Chapter 14
Manipulating Raster Data

To display a raster layer in ArcMap, we can call the RasterLayer co-class to instantiate a raster layer object. But the new raster layer object is an empty shell. To add data into the layer, we need to go through a few steps. First, we use the RasterWorkspaceFactory co-class to instantiate a workspace factory object. Second, we call the OpenFromFile() method on the IWorkspaceFactory to create a workspace object. Third, we call the OpenRasterDataset() method on the IRasterWorkspace interface of the workspace object to create a raster dataset object; and finally we pass the raster dataset object to a raster layer object (Figure 1). In this process, the raster dataset is the core of concern.

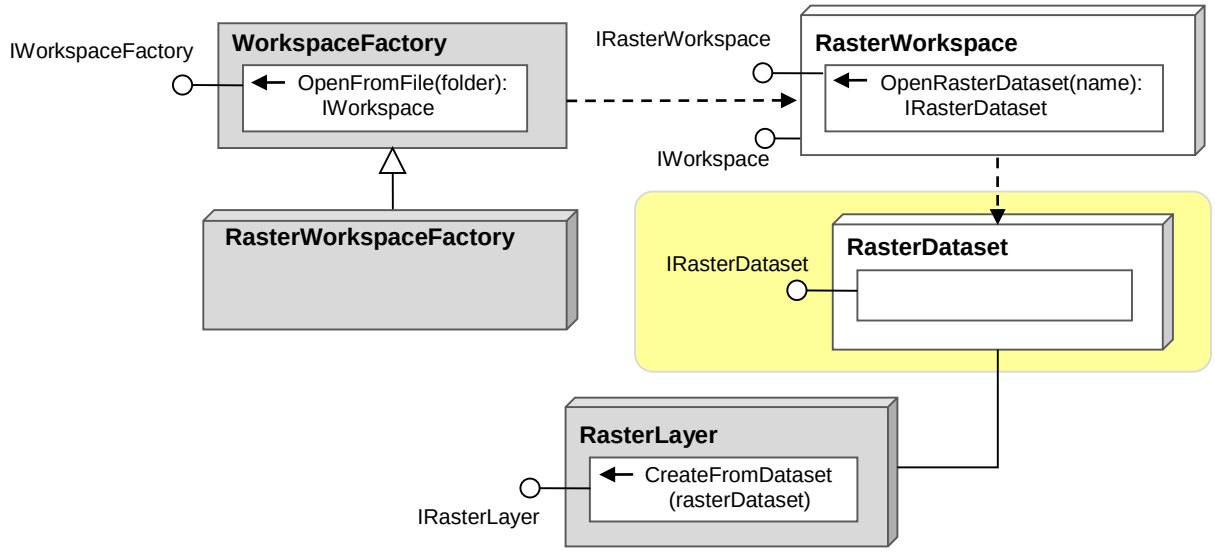

Figure 1 Components involved in creating a raster dataset
(See Page 1 of DataSourcesRasterObjectModel.pdf and Page 3 of CartoObjectModel.pdf)

A raster dataset object represents a set of raster data stored in a file system or in geodatabase. The RasterDataset component has many properties and methods on various interfaces to describe the raster or carry out raster operations. For example, it has properties that tell us the raster format, extent, spatial reference, and number of bands of a raster. It has methods to let us copy, rename, and delete raster datasets, to build color maps, as well as to build pyramids to improve display performance for large datasets. A raster dataset object also provides methods to merge pixels from other raster datasets into itself.

This chapter will introduce another important ArcObjects component Raster which is used to manipulate raster data. While raster dataset is what stored on hard drive, a raster object is the pixel data loaded into the memory. Therefore a raster object is convenient for displaying, rendering, on-the-fly coordinate transformation, as well as data analyzing. Since a raster resides in the memory, it is a transient object. Any change or transformation made to a raster is not automatically saved to the raster dataset on the hard drive.

This chapter will analyze object model diagrams for raster related components and use examples to demonstrate how to read pixel values, create new raster datasets, and perform analysis on raster data.

1. Identifying pixel values

Example 1: Create a raster identify tool to display pixel values.

To find the value of a pixel in a raster layer, we need to use the raster object. A raster object is similar to a feature class object in that both are data loaded into the memory and both can be set to and get from a layer object, although a feature class is related to a vector layer while a raster related to a raster layer. Unlike FeatureClass which is a regular class, the Raster class is a co-class so that it can instantiate new raster objects in certain situations. A raster object can also be obtained from a raster layer in a map document in ArcMap (Figure 2). A raster object provides methods for getting pixel values, adding pixel data such as bands and pixel blocks, as well as saving itself as a raster dataset on the hard drive.

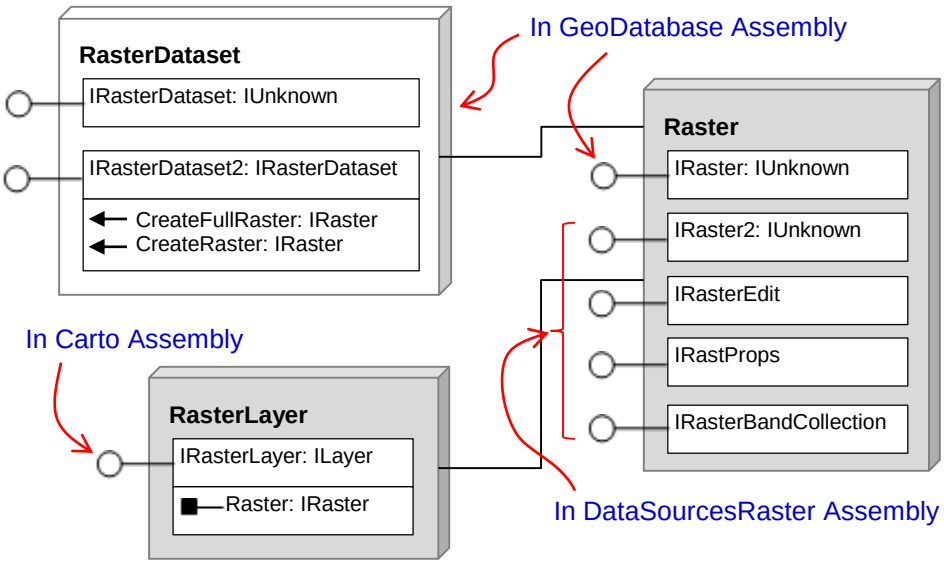

Figure 2 Ways of obtaining raster objects

Suppose a map document contains a raster layer, we can get a raster object from the Raster property on the IRasterLayer interface of the raster layer. Hopping from the map document to a raster object is similar to hopping from the map document to a feature class in the case of feature layers.

The Raster class has many interfaces that can be freely switched from one to another by Query Interface (QI). Figure 2 lists five frequently used interfaces which are managed in different ArcObjects assemblies. Interface IRaster2, which is contained in the DataSourcesRaster assembly, provides a method named GetPixelValue() that gets a pixel value at a given column and row (Figure 3). In addition, this interface contains four methods for converting between map coordinates and pixel columns and rows.

The GetPixelValue() method takes three arguments: a band index (top band: 0), a column number, and a row number. It returns a value defined by the pixel type. For example, an image

usually has three bands (red, green, and blue), each containing integer values between 0 and 255. A DEM or slope gradient raster usually has only one band containing single-precision floating numbers (Single).

Since we can get the mouse cursor location from the CurrentLocation property of the map document, we can call the ToPixelColumn() and ToPixelRow() methods on the IRaster2 interface to convert the map coordinates of a clicking point into pixel columns and rows. Finally we can call the GetPixelValue() to get a pixel value of a specific band.

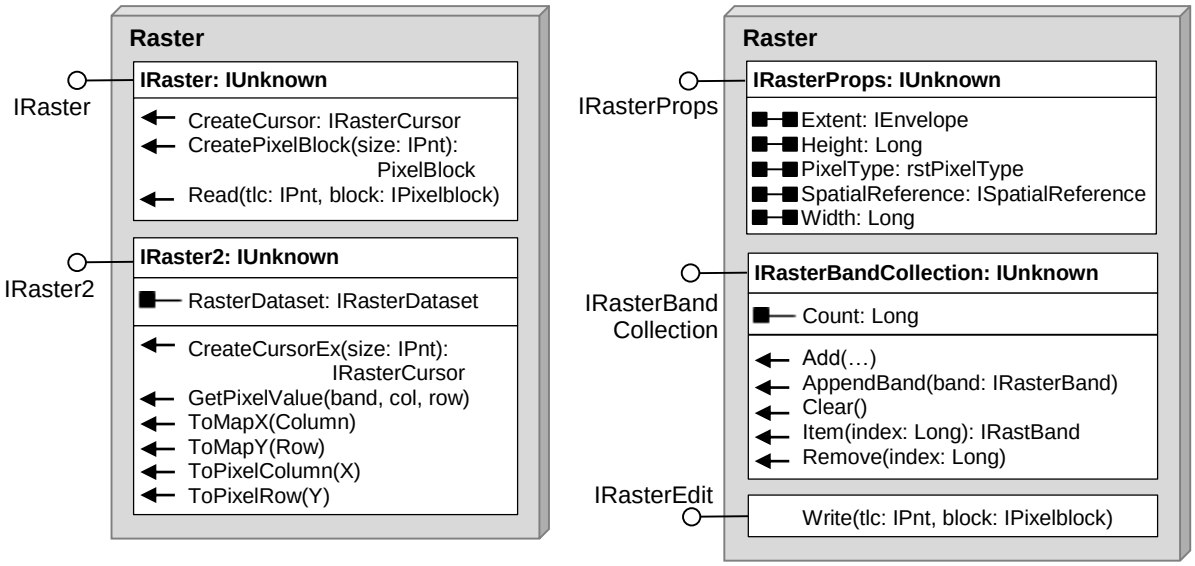

Figure 3 A few important interfaces of the Raster Class
(See details in DataSourcesRasterObjectModel.pdf)

Please perform the following steps to implement a pixel value identify add-in tool:

Step 1: Create a new ArcMap Add-in project named “RasterAnalyst” (icon in folder Images on CD-ROM). In the project, create an add-in tool named “RasterIdentifyTool” (file name: “RasterIdentifyTool.vb”, icon , caption and tooltip text: “Idenitfy”). Then create a toolbar with caption “Raster Analyst Toolbar”, and add the raster idnetify add-in tool into the toolbar.

Step 2: Add ArcGIS assemblies Carto, Geodatabase, DataSourceRaster, and Geometry into the project as references. Then add the following import statements in the RasterIdentifyTool class module.

```
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Carto
Imports ESRI.ArcGIS.DataSourcesRaster
Imports ESRI.ArcGIS.Geometry
```

Step 3: Bring up the OnMouseDown event procedure of the identify tool.

```
Protected Overrides Sub OnMouseDown(arg As
                                    ESRI.ArcGIS.Desktop.AddIns.Tool.MouseEventArgs)
    MyBase.OnMouseDown(arg)
```

```
End Sub
```

Step 4: In the event procedure, hop from the map document to the raster object.

```
Dim mxDoc As IMxDocument = My.ArcMap.Document    ' gets the document
Dim lyr As ILayer = mxDoc.SelectedLayer          ' gets the active layer
' Note: you may add code here to check if lyr is Nothing (when no layer is selected)

' try to cast the layer to a raster layer
Dim rlyr As IRasterLayer = rlyr = TryCast(lyr, IRasterLayer)
' Note: you need to add code here to check if rlyr is nothing
' which happens when the selected layer is not a raster layer

Dim raster As IRaster2 = rlyr.Raster    ' gets the raster from the raster layer
```

Step 5: Get the mouse click point by calling the CurrentLocation method on the IMxDocument interface of the map document object.

```
Dim p As IPoint = mxDoc.CurrentLocation
```

Step 6: Convert the x and y coordinates of the click point (in map units) to pixel columns and rows.

```
Dim pixcol As Integer = raster.ToPixelColumn(p.X)
Dim pixrow As Integer = raster.ToPixelRow(p.Y)
' Note: you will add code here to check if column and row are within raster extent
```

Step 7: Get the pixel value of a specific band and report result. To get the top band, for example,

```
' Note: you will add code here to check if the pixel carries NoData value
Dim str As String = raster.GetPixelValue(0, pixcol, pixrow).ToString
MsgBox("Pixel value: " & str)
```

This tool should work if it is properly used: 1) a raster layer is selected in ArcMap; 2) you click inside the raster extent; 3) you click on a non-NoData cell. If any of these conditions is not satisfied, the program can go wrong. To strengthen the program, you need to add code in step 4 to check whether a raster layer is selected (you should be able to accomplish this task). In step 6, you need to check if the pixel column and row are within extent of the raster, and in step 7, you need to check if the pixel carries a NoData value before converting it to string for reporting.

To check if the clicked column and row numbers are within the raster extent, you need to know the total number of columns and rows of the raster. These values can be obtained from the Width and Height on the IRasterProps interface of the raster object. Therefore, you can add the following code in step 6.

```
Dim rstprops As IRasterProps = raster     ' QI to the IRasterProps interface
If pixcol < 0 Or pixrow < 0 Then Return
If pixcol > rstprops.Width Then Return
If pixrow > rstprops.Height Then Return
```

To enhance step 7, if a pixel carries a NoData value, the GetPixelValue() method returns a Nothing object, therefore you may use the following code for this step to avoid converting object Nothing to a string.

```
Dim str As String
If raster.GetPixelValue(0, pixcol, pixrow) = Nothing Then
    str = "Nodata."
Else
    str = raster.GetPixelValue(0, pixcol, pixrow).ToString
End If
MsgBox("Pixel value: " & str)
```

After the above enhancement, the tool will never crash. However, if the raster contains multiple bands, it only displays the top band. To display values on all bands at the click point, step 7 can be further improved. First, you can use the IRasterBandCollection interface of the raster object to get the number of bands. Once you know the number of bands, you can use a loop to get the value on each band. Please replace the code in step 7 by the following code to make your raster identify tool work like a professional product.

```
Dim bands As IRasterBandCollection = raster      ' QI
Dim str As String = "Pixel (" & pixrow & ", " & pixcol & ") values:"
For i As Integer = 0 To bands.Count - 1
    If raster.GetPixelValue(i, pixcol, pixrow) = Nothing Then
        str += Chr(10) & "    Nodata."
        Exit For
    Else
        str += Chr(10) & "    Band " & (i + 1).ToString & ": " &
            raster.GetPixelValue(i, pixcol, pixrow).ToString
    End If
Next
MsgBox(str, MsgBoxStyle.Information, "Identify")
```

2. Creating new raster datasets

Example 2: Create a new raster dataset from scratch and assign values to raster cells.

Before creating a new raster dataset, this section will introduce a few key concepts: raster dataset, raster, pixel block, and pixel data.

2.1 Raster dataset

The Workspace class has several methods on various interfaces that allow us to create new raster datasets on the disk or in a geodatabase as well as to write pixel values into them. The IRasterWorkspace interface (introduced in Chapter 10) has an OpenRasterDataset() method that allows us to open (load) an existing raster dataset. In the object model diagram below, you can find that the IRasterWorkspace2 interface has a CreateRasterDataset() method that creates new raster datasets.

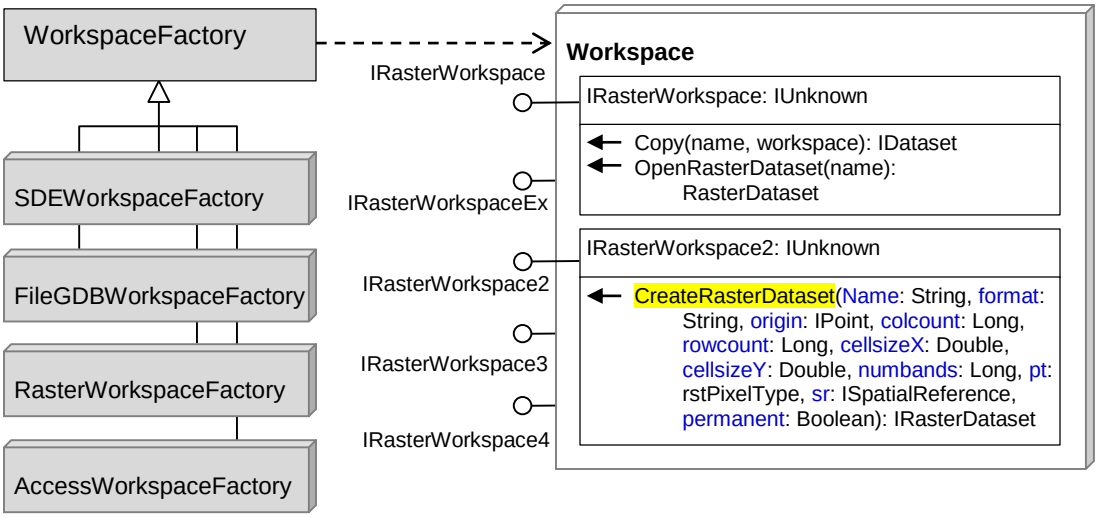

Figure 4 The Workspace regular class

A workspace object is needed in order to create a new raster dataset. Depending on how a new raster dataset will be stored, a proper workspace factory must be selected to create the workspace. In this example, you will use the RasterWorkspaceFactory to create a workspace in the Windows file system. If you want to create a raster dataset in a file geodatabase, you can use FileGDBWorkspaceFactory. Once you obtain a workspace object, you can call the CreateRasterDataset() method on the IRasterWorkspace2 interface to create new raster datasets.

The CreateRasterDataset() method requires 11 parameters:

1) name: String — A string name for the new raster dataset, must not contain spaces and certain special characters
2) format: String — Options: "GRID", "TIFF", "IMAGINE Image", "BMP", "RST" (Idrisi format), and "MEM" (in-memory raster). All are case-sensitive.
3) origin: IPoint — The coordinates of the lower-left corner of the new raster
4) colcount: Long — Number of columns, integer
5) rowcount: Long — Number of rows, integer
6) cellsizeX: Double — Pixel width
7) cellsizeY: Double — Pixel height
8) numbands: Long — Number of bands
9) pxltype: rstPixelType — Built-in constants for pixel data types
10) sr: ISpatialReference — Spatial reference for the new raster
11) permanent: Boolean — Whether to permanently store the raster

To create a new raster dataset, you must prepare values for all these arguments. Although the parameter list is long, the first eight and the last parameter are straightforward. For the tenth parameter (spatial reference), you can create a spatial reference object using the GDBManager class introduced in Chapter 13. For the ninth parameter (raster pixel data type), you can select one from an enumeration of constants for raster pixel data types (Table 1). In VB programming, the IntelliSense displays a list of constants when an argument for this parameter is needed.

Table 1 ESRI raster pixel type constants

Constant	Value	Description

PT_UNKNOWN	-1	Pixel values are unknown.
PT_U1	0	Pixel values are 1 bit. Can store binary values 0 and 1
PT_U2	1	Pixel values are 2 bits. Can store 4 different values
PT_U4	2	Pixel values are 4 bits. Can store 16 different values
PT_UCHAR	3	Pixel values are unsigned 8 bit integers. For 0 to 255
PT_CHAR	4	Pixel values are 8 bit integers. For -127 to +128
PT_USHORT	5	Pixel values are unsigned 16 bit integers. For 0 to 65536
PT_SHORT	6	Pixel values are 16 bit integers. For -32767 to +32768
PT_ULONG	7	Pixel values are unsigned 32 bit integers. For unsigned Long
PT_LONG	8	Pixel values are 32 bit integers. For signed Long
PT_FLOAT	9	Pixel values are single precision floating point.
PT_DOUBLE	10	Pixel values are double precision floating point.
PT_COMPLEX	11	Pixel values are single precision complex.
PT_DCOMPLEX	12	Pixel values are double precision complex.
PT_CSHORT	13	Pixel values are short integer complex.
PT_CLONG	14	Pixel values are long integer complex.

(Source: ESRI ArcObjexts API reference for .NET)

Once you have determined values for all the 11 arguments, you can use the following example to create a blank raster dataset of the ESRI grid raster format:

```
Dim rasterDS As IRasterDataset2 = ws.CreateRasterDataset(
    FileName, "GRID", origin, width, height, cellSizeX, cellSizeY,
    numBand, pxlType, sr, True)
```

2.2 Raster and pixel block

Once you create a raster dataset, you can obtain a raster object from it by calling the CreateFullRaster() method on the IRasterDataset2 interface (Figure 2). If your raster dataset object points to the IRasterDataset interface, a QI needs to be performed. Without requiring any argument, the CreateFullRaster() method returns a raster object (pointing to the IRaster interface) containing all bands of the dataset. The IRasterDataset2 interface also provide a CreateRaster() method (Figure 2). But this method creates an empty raster whose width, height, and number of bands are 0. Therefore, you are not suggested to use the CreateRaster() method in this example.

```
'Create a raster from the raster dataset.
Dim raster As IRaster = rasterDS.CreateFullRaster()
```

A new raster object created by the code above is empty and it must be populated with pixels. A raster dataset lays out a structure for permanent data storage, and a raster object lays out a structure in the memory for loading or manipulating pixel data. To manipulate pixel data in these structures, you need to understand another ArcObjects component named pixel block (Figure 5).

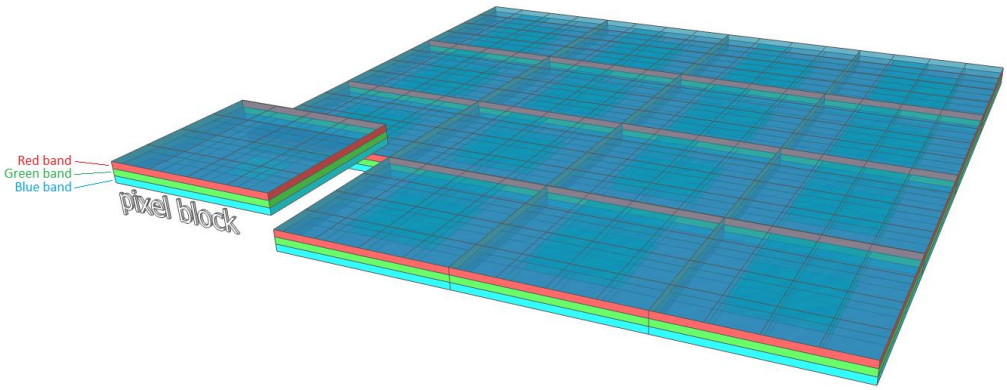

Figure 5 Raster, pixel blocks, and pixels

A pixel block is a block of pixels used to facilitate access to pixel data in large rasters. The size (rows and columns) of a pixel block is specified by the user, and a pixel block does not have to be a square. In practice, a size with the same number columns as the raster and a smaller row number is often used. If a raster is not too large, and the entire raster can be handled by the computer's memory capacity, a pixel block with same size as the entire raster can be used. In a pixel block, each band is a two-dimensional array of pixel values. We can read pixel values from these arrays, modify them, and save the pixel block back to the raster dataset. Figure 6 explains how pixel block objects can be derived from a raster object.

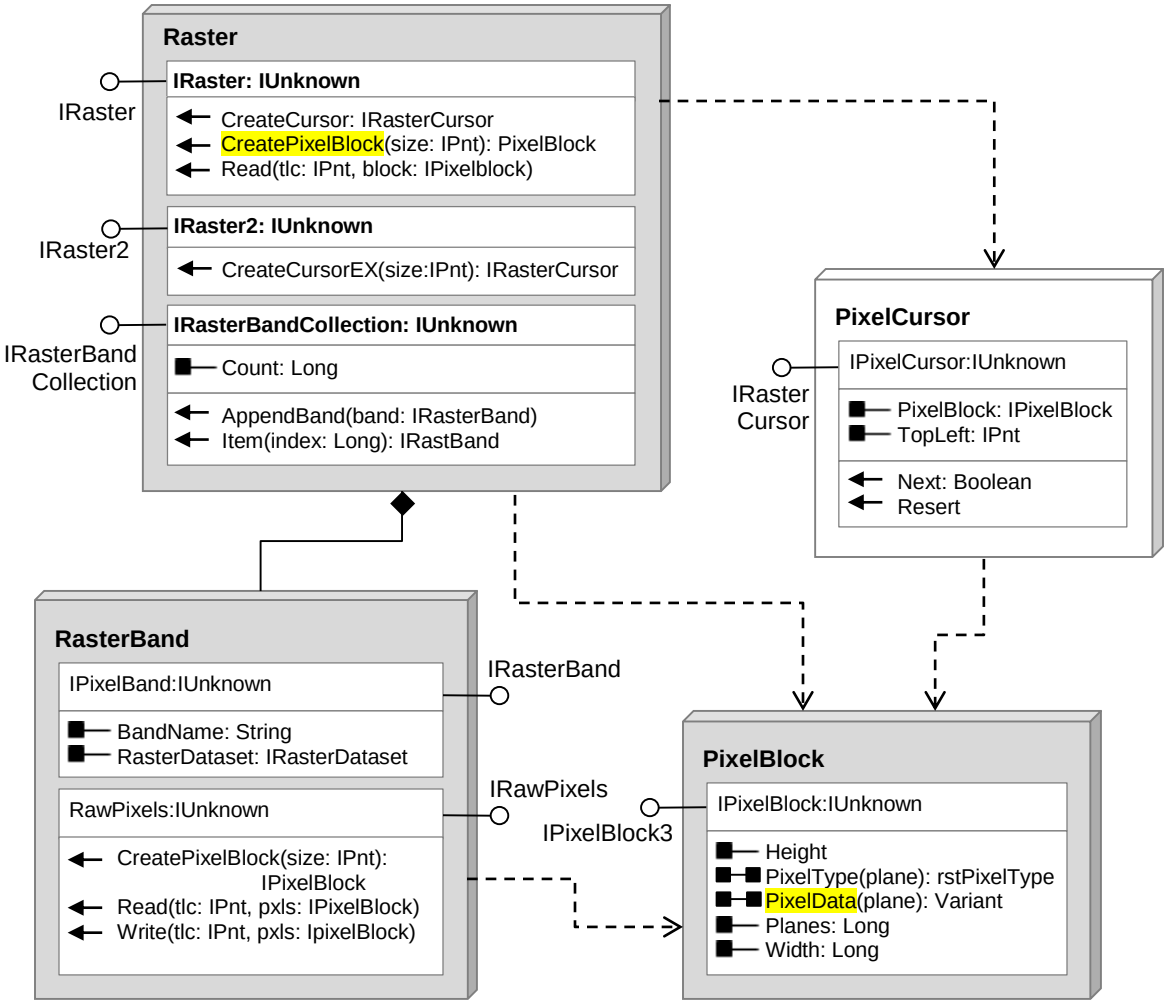

Figure 6 Getting pixel blocks from a raster
(Details in DataSourceRasterObjectModel.pdf)

There are many ways to create a new or retrieve a pixel block from a raster. With whatever method, you need to specify a size for the block. What is interesting about this size argument required by various methods is that it is a Pnt (point) object. The Double type X and Y properties of the Pnt object are used to represent width and height of pixel blocks. You can instantiate a size object from the Pnt class as the following:

```
Dim size As IPnt = New Pnt
size.SetCoords(width, height)  ' variables width, height must carry values
```

The second line above does the same job as the following two lines:

```
size.X = width
size.Y = height
```

To create a new pixel block to store data for a new raster, you can call the CreatePixelBlock() method on the IRaster interface, which returns a pixel block object with IPixelBlock interface.

```
Dim pixelBlock As IPixelBlock = raster.CreatePixelBlock(size)
```

A pixel block object is composed a set of bands, also called planes, each is a two-dimensional array of pixel values. Please recall that one-dimensional arrays are introduced in Chapter 5. A two-dimensional array is similar to a one-dimensional array but it carrying a matrix (rows and columns) of value of one data type. To access a value in a two-dimensional array requires a row index and a column index. The IPixelBlock3 interface of a pixel block object has a two-way (read and write) PixelData property. You can obtain a two-dimensional safe array for a specific band from the PixelData property by providing a band index. You can also set a two-dimensional array object to this property to update a specific band of a raster dataset. Normally when you declare an array you need to specify a data type such as integer or single. A safe array can store any type of values without having to specify a data type. Although a safe array looks convenient since you do not need to specify a data type, as a user, however, you have to figure out the correct type of values obtained from a safe array. The following example gets an array of pixel values of the top band (index: 0) from a pixel block.

```
Dim pixels As System.Array  ' declares a safe array variable for any data type
pixels = CType(pixelBlock.PixelData(0), System.Array)
```

Once you get a pixel array, you can write value into or read values from it by using two nested For … Next repetitions. Suppose you know that pixel values are integer, for example,

```
Dim value As Integer
For col As Integer = 0 To <pixelBlockWidth> - 1       ' iteration for columns
    For row As Integer = 0 To <pixelBlockHeight> - 1  ' iteration for rows
        ' to read a value from pixel (col, row)
        value = pixels.GetValue(col, row)

        ' or to set a value to pixel (col, row)
        pixels.SetValue(value, col, row)
        ' or simply
        pixels(col, row) = value
    Next row
Next col
```

If you update values in a pixel array, you can write the array back into the pixel block by setting the array to the PixelData property with a specific band index. For example,

```
pixelBlock.PixelData(0) = pixels
```

To write an updated pixel block into the raster dataset so as to make it permanent (save it to disk), you can call the Write method on the IRasterEdit interface of the raster object (see Figure 3). The Write method has two parameters: the pixel block to save and a position at which the top-left corner of the pixel block will be placed in the raster. The position parameter requires a Pnt object whose X and Y properties represent the column and row in the raster respectively (Figure 7). If a pixel block covers the entire raster, a position argument must be set with coordinates (0, 0). For example,

```
Dim tlc As IPnt = New Pnt      ' declares variable for top-left corner
tlc.SetCoords(0, 0)            ' sets column and row as 0, 0
```

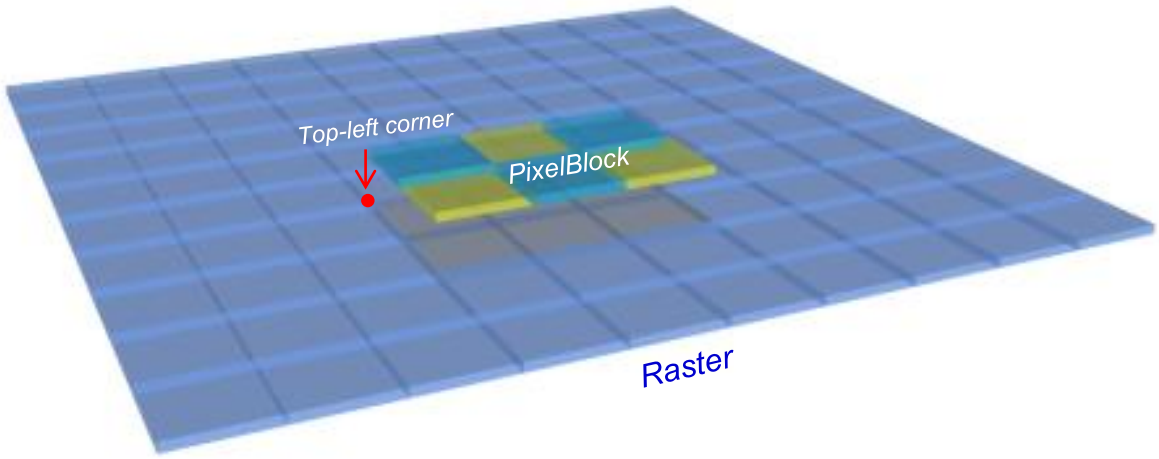

Figure 7 Writing a pixel block to a raster

Finally, to save the pixel block, you need to perform a QI to get the IRasterEdit interface of the raster object and then call its Write() method (see Figure 3).

```
Dim rasterEdit As IRasterEdit = raster  ' QI to the IRasterEdit interface
rasterEdit.Write(tlc, pixelBlock)       ' writes the pixel block to raster
```

2.3 Implementation

Now, you are ready to create an add-in button that will be used to create a test raster dataset and populate it with some arbitrary pixel values.

Step 1: Create an add-in button in your RasterAnalyst project. Name it “CreateRasterButton” with caption and tooltip text “Create Raster” and use icon  (in folder Images on CD-ROM). After creating the button, add it into the Raster Analyst toolbar in the project.

Step 2: Add the following import statements in the CreateRasterButton class module:

```
Imports ESRI.ArcGIS.DataSourcesRaster
Imports ESRI.ArcGIS.Geodatabase
Imports ESRI.ArcGIS.Geometry
```

Step 3: Create a blank custom sub procedure as the following:

```
Public Sub CreateRasterDataset(ByVal path As String, ByVal rasterName As String,
           format As String, origin As IPoint, width As Integer, height As Integer,
           cellsizeX As Double, cellsizeY As Double, numBands As Integer,
           pxlType As rstPixelType, sr As ISpatialReference,
           permanent As Boolean)

End Sub
```

Step 4: Add the following code inside the new sub procedure to validate user input and create workspace. Please note that the colon symbol between message box and keyword Return allows us to write two lines of code in one line.

```
' ================================================
' Part 1: open a raster workspace
' ================================================
' check existence of user input path. Path must exist
If Not IO.Directory.Exists(path) Then MsgBox("Path does not exist.") : Return

' check existence of raster. Raster of given name must not exist
If IO.Directory.Exists(path & "\" & rasterName) Then
    MsgBox("Raster already exists.") : Return
End If

' create a workspace factory from the RasterWorkspaceFactory coclass
Dim workspaceFact As IWorkspaceFactory = New RasterWorkspaceFactory
' create a workspace object from user-specified folder
Dim ws As IRasterWorkspace2 = workspaceFact.OpenFromFile(path, 0)

If ws Is Nothing Then
    MsgBox("Cannot open raster workspace '" & path & "'") : Return
End If
```

Step 5: Add the following block of code below part 1. This is a key step to actually create a blank raster dataset by calling the CreateRasterDataset() method on the IRasterWorkspace2 interface. The variable (rasterDS) declared in the first line directly points to interface IRasterDataset2 because you will not need to use the interface IRasterDataset returned by the method. To avoid accidental crash, the method call is placed in a Try block.

```
' ================================================
' Part 2: create a new raster dataset
' ================================================
Dim rasterDS As IRasterDataset2   ' declare a variable for raster dataset

Try     ' use a Try block to safely call the CreateRasterDataset method
    rasterDS = ws.CreateRasterDataset(rasterName, format, origin,
                        width, height, cellsizeX, cellsizeY,
                        numBands, rstPixelType.PT_UCHAR, sr, True)
Catch ex As Exception
    MsgBox("Creating raster dataset failed.") : Return
End Try
```

Step 6: Add the following code below code of part 2. This part first creates a raster object from the raster dataset, then it defines 255 as NoData value in the top band (the only band), creates a pixel block that covers the entire raster, assigns an arbitrary value to each pixel, and finally saves the pixel block to the raster dataset. Please refer to Figure 6 while reading the code below.

```
' ================================================
' Part 3: create a raster and write values in it
' ================================================
' 1) Create a raster from the dataset
Dim raster As IRaster = rasterDS.CreateFullRaster
```

```
' 2) Get the only raster band to specify NoData pixel value
Dim bands As IRasterBandCollection = raster     ' QI to IRasterBandCollection
Dim band As IRasterBand = bands.Item(0)         ' gets the top band
Dim rasterProps As IRasterProps = band          ' QI to IRasterProps of band 0
' set NoDataValue property, value 255 will represent NoData for this band
rasterProps.NoDataValue = 255

' 3) create a blocksize object using the Pnt class, interesting!
Dim blockSize As IPnt = New Pnt
blockSize.SetCoords(width, height)  ' assigns width and height to X and Y!
Dim pixelBlock As IPixelBlock3 = raster.CreatePixelBlock(blockSize)

' 4) Populate the pixel block with arbitrary values.
' get the 2-D pixel array from the top band of the pixel block
Dim pixels As System.Array = CType(pixelBlock.PixelData(0), System.Array)
For col As Integer = 0 To width - 1         ' iterates through each column
    For row As Integer = 0 To height - 1    ' iterates through each row
        If col = row Then
            pixels.SetValue(255, col, row)  ' sets NoData to pixels on the diagonal
        Else
            ' set an arbitrary number to pixel (col, row)
            pixels.SetValue(Convert.ToByte((col * row) / 255), col, row)
        End If
    Next row
Next col

' 5) set the pixel array back to the PixelData property of the pixel block
pixelBlock.PixelData(0) = pixels    ' 0 is band index

' 6) define where in the raster to place the pixel block (its upper-left pixel)
Dim upperLeft As IPnt = New Pnt
upperLeft.SetCoords(0, 0)   ' (0, 0) is the upper-left corner of the raster

' 7) write the pixel block into the raster dataset
Dim rasterEdit As IRasterEdit = raster    ' QI to IRasterEdit interface
rasterEdit.Write(upperLeft, pixelBlock)

' 8) release memory occupied by the raster
System.Runtime.InteropServices.Marshal.ReleaseComObject(rasterEdit)

MsgBox("New raster dataset has been successfully created!", MsgBoxStyle.Exclamation)
```

Step 7: In the OnClick event procedure of the Create Raster button, prepare values for all arguments required by sub procedure CreateRasterDataset(), and finally call the sub procedure to create a new test raster dataset. For example,

```
Dim path As String = "C:\Temp"          ' folder the new dataset will be created in
Dim rstName As String = "TestRaster"    ' name of new raster
Dim format As String = "GRID"           ' to create a raster in ESRI grid format
Dim origin As IPoint = New Point        ' coordinates of the lower-left corner
                                          ' of the new raster
origin.PutCoords(15, 15)                ' just assign an arbitrary coordinate
Dim width As Integer = 100              ' number of columns
Dim height As Integer = 100             ' number of rows
Dim cellsizeX As Double = 10            ' cell size in X direction
Dim cellsizeY As Double = 10            ' cell size in Y direction
Dim numBands As Integer = 1             ' number of bands
```

```
Dim pxlType As rstPixelType = rstPixelType.PT_UCHAR
                                          ' pixel data type: 8 bit integer (0 - 255)
Dim sr As ISpatialReference = New UnknownCoordinateSystem  ' spatial reference
Dim perm As Boolean = True            ' yes, save the new raster on hard drive

' call the CreateRasterDataset procedure to create new raster dataset
CreateRasterDataset(path, rstName, format, origin, width, height,
                    cellsizeX, cellsizeY, numBands, pxlType, sr, perm)
```

The CreateRasterDataset requires 12 arguments! This is probably the longest list of arguments passed by a procedure you have ever seen. Please make sure the output folder (C:\Temp or whatever you choose) exists. When you pass these arguments to the procedure, please pay attention to the order of them.

Finally, you can test the create raster button. If successful, you can manually load the resulting raster into ArcMap to display. After properly stretching the raster, the layer should look similar to Figure 8. The white pixels on the diagonal, those are pixels purposely assigned with NoData (255 in this example).

By passing proper arguments to parameters of function CreateRasterDataset(), you can create raster datasets with any spatial reference, any extent, any number of bands, any pixel size, as well as any pixel data type. By modifying the code of this example, you may also create a raster dataset in certain none-ESRI formats including TIFF, BMP, Erdas Imagine, and Idrisi RST, and you may store the result in any ESRI supported geodatabase including SDE GDB, file GDB, or personal GDB.

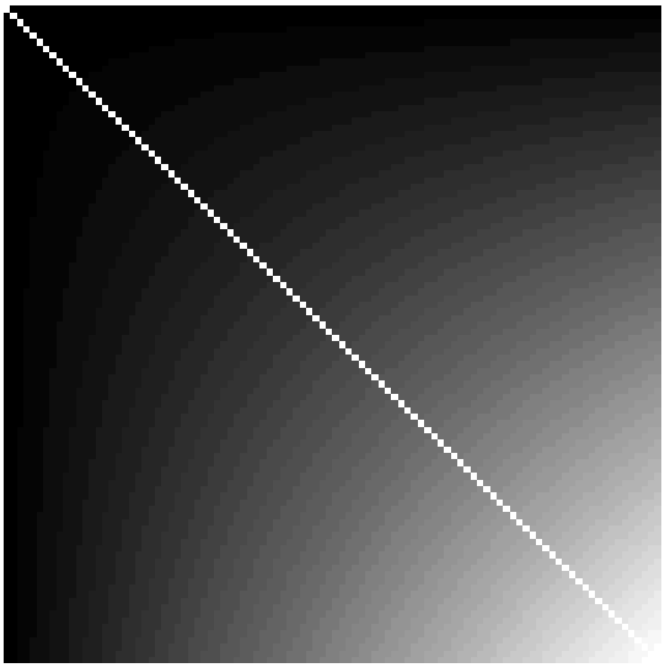

Figure 8 Resulting new raster

3. Local Function – Raster reclassification

Example 3: Create a raster classification add-in button that reclassifies a slope raster from degrees into classes using the following table.

Class	Percentage	Degrees	Description
1	≤ 2%	≤ 1.15	Flat
2	2 – 5%	1.15 – 2.86	Gently undulating
3	5 – 8%	2.86 – 4.57	Undulating
4	8 – 15%	4.57 – 8.53	Rolling
5	15 – 30%	8.53 – 16.70	Moderately steep
6	> 30%	> 16.70	Steep

3.1 Object model analysis and solution development

Raster reclassification is to convert pixel values from one scale to another. It is considered a local function since only one pixel is involved in each classification computation. Generally, a raster reclassification can be accomplished by the following procedure.

1) Make a copy of the input raster dataset
2) Get a raster object from the new raster dataset
3) Create a raster cursor from the raster
4) Iterate through each pixel block in the raster cursor
5) Get a pixel array from a pixel block
6) Reclassify each pixel value in the pixel array
7) Update the pixel block with the reclassified pixel array
8) Save the pixel block back into the raster

So far, you have learned most of the ArcObjects components and techniques required for this tool. You have known raster dataset, raster, raster block, and pixel array, and you are able to move from one object to another by calling proper properties and methods (see Figures 2 and 6). You can find sample code to carry out step 2) and steps 5) through 8) in Example 2. In order to accomplish all steps of the procedure, the following will introduce one new interface of the raster class, the ISaveAs interface, and one new class the RasterCursor class.

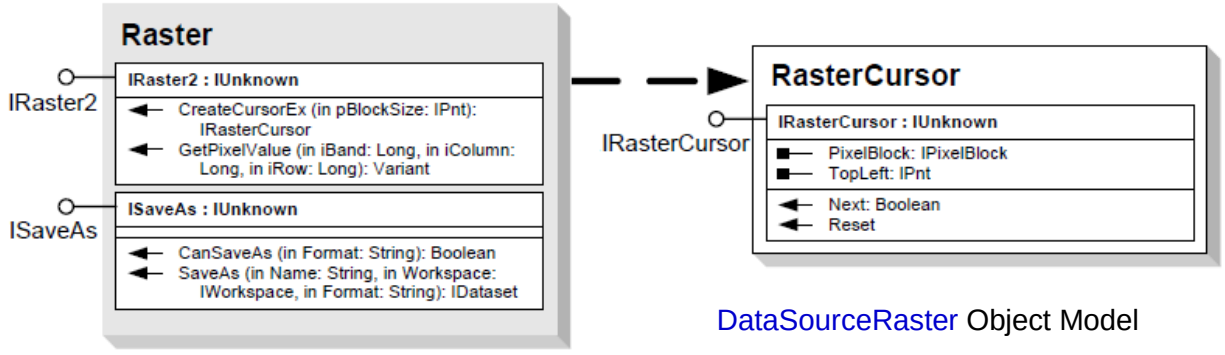

Figure 9 Raster and RasterCursor object model

To make a copy of an existing raster dataset, you can call the SaveAs() method on the ISaveAs interface of the Raster class. The SaveAs method has three parameters: a new raster name as string, a workspace object, and the target raster format in string (Figure 9). To get a workspace object for the second argument, you need to use an appropriate workspace factory co-class to instantiate a workspace factory object. Given a workspace (the full path of a folder), you can call the OpenFromFile method on the IWorkspaceFactory interface to get a workspace object (see Figure 1). The third argument (the raster format) is a string for an ESRI supported format including "GRID", "TIFF", IMAGINE Image", "BMP", and "RST". To check if a format is supported by the SaveAs method, you can call the CanSaveAs() method on the same interface to test if a specific format is supported in your system. The CanSaveAs() method takes a format string as input and returns a Boolean result true or false. For example, if you are provided with a raster object represented by variable *inRaster* pointing to IRaster interface, an output workspace represented by variable *outFolder* (which carries the full path to a folder), an output raster name represented by variable *outRasterName* (which carries an output name), as well as an intended raster format represented by variable *format* (suppose it carries "GRID"), you can save the raster to the new workspace with the provided name as the following.

```
' get the output workspace
Dim wksFact As IWorkspaceFactory = New RasterWorkspaceFactory
Dim outWS As IRasterWorkspace2 = wksFact.OpenFromFile(outFolder, 0)

' QI to the ISaveAs2 interface of the input raster
Dim rasterSaveAs As ISaveAs2 = inRaster     ' QI

' to save input raster dataset to output location
Dim outRasterDS As IRasterDataset2 =
    rasterSaveAs.SaveAs(outRasterName, outWS, outFormat)
```

With a new raster dataset object outRasterDS pointing to IRasterDataset2, you can obtain an output raster by calling the CreateFullRaster() method by the following line.

```
Dim outRaster As IRaster2 = outRasterDS.CreateFullRaster()
```

In Step 3) of the procedure, you need to get a raster cursor object from the raster. A raster cursor is a collection of pixel blocks. Suppose you have a raster, you can call the CreateCursorEx() method on the IRaster2 interface to cut the raster with a given size. The result of the method is a raster cursor containing a collection of pixel blocks cut out of the raster (Figure 9). The only argument the method require is a Pnt (a point) object to indicate the size of pixel blocks to cut. The X and Y properties of the Pnt object represent the number of columns and rows of pixel blocks. The following example demonstrates how to create a raster cursor from a raster.

```
Dim size As IPnt = New Pnt    ' creates a new Pnt object for pixel block size
size.SetCoords(100, 5)        ' 100 columns and 5 rows

Dim RasterCursor As IRasterCursor = outRaster.CreateCursorEx(size)
```

The raster cursor object has one interface IRasterCursor (Figure 9). This interface has a PixelBlock property that returns a current pixel block object. The TopLeft property on the IRasterCursor interface returns a Pnt object that indicates the column and row position of the top-left corner of the current pixel block in the entire raster. To iterate through all pixel blocks,

you can repeatedly call the Next method. If a pixel block is available, the Next method moves a pointer to the next pixel block and returns a Boolean value True. Otherwise, the Next method returns value False. The Reset method restores the pointer to the first pixel block. The following code shows how to use a Do … Loop repetition to iterate through all pixel blocks in a raster cursor:

```
Dim pixelBlock As IPixelBlock3              ' declares a pixel block variable
Do
    pixelBlock = RasterCursor.PixelBlock    ' gets a pixel block from cursor
    ' process data in the pixel block ...
Loop While RasterCursor.Next                ' moves to next pixel block
```

Steps 5) through 7) which process pixel data can be carried out inside this repetition block as soon as a pixel block is obtained. Suppose you have declared a safe array variable *pixels*, you can get an array of pixels of a specific band from the PixelData property of the pixel block, i.e.

```
pixels = CType(pixelBlock.PixelData(0), System.Array)  ' 0: index of the top band
```

With a safe array, its columns and rows are unknown and they are determined by the pixel block size. Although you specified a pixel block size when you cut a raster into a raster cursor, not all resulting pixel blocks in the cursor are of the exact specified size. Blocks on the right and bottom edges are often partial because the size you specify may not divide the raster width and height into whole numbers. It is similar to installing a tile floor. When you start from one side of the room, you often have to cut tiles when you reach the opposite wall. The actual size of a pixel block can be derived from the Width and Height properties of each resulting pixel block object. Given a custom function Reclassify() that converts an input value into another value, the following shows how to use two nested For … Next repetitions to iterate through all pixels in the pixel array and reclassify each pixel.

```
For col As Long = 0 To pixelBlock.Width - 1        ' iterates through columns
    For row As Long = 0 To pixelBlock.Height - 1   ' iterates through rows
        value = pixels.GetValue(col, row)          ' reads a pixel value
        ' perform reclassification, store result back into the array
        pixels(col, row) = Reclassify(value)   ' Reclassify() is a custom function
    Next
Next
```

After updating all pixel values in the array, you can set the updated array back into the pixel block.

```
pixelBlock.PixelData(0) = pixels         ' 0 – band index, top band
```

Finally, after updating all bands of a pixel block, you can write the pixel block back into the raster to make the change permanent on the hard drive.

```
Dim outRasterEdit As IRasterEdit = outRaster  ' QI to the IRasterEdit interface
Dim tlc As IPnt = RasterCursor.TopLeft ' gets the position of the current pixel block
outRasterEdit.Write(tlc, pixelBlock)   ' writes the pixel block
```

3.2 Implementation

Please find a slope raster in ESRI grid format in folder Data on the CD-ROM. Please use ArcCatalog to copy this raster dataset to a workspace (folder) on the hard drive. Slope values in the raster are degrees. You will create an add-in button in the RasterAnalyst project to reclassify this slope raster into six classes using the table above.

Step 1: Create an ArcMap add-in button named "ReclassifySlopeButton" in your RasterAnalyst project. Use caption and tooltip text "Reclassify Slope" and icon  (in folder Images on CD-ROM) for the button. Add the button into the project toolbar.

Step 2: Inside the ReclassifySlopeButton class, add the following custom function that reclassifies slope values:

```
Private Function Classify(value As Double) As Integer
    Dim result As Integer
    Select Case value
        Case Is <= 1.15
            result = 1
        Case Is <= 2.86
            result = 2
        Case Is <= 4.57
            result = 3
        Case Is <= 8.53
            result = 4
        Case Is <= 16.7
            result = 5
        Case Else
            result = 6
    End Select
    Return result
End Function
```

Step 3: Add the following import statements on top of the Button_Reclassify class.

```
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.DataSourcesRaster
Imports ESRI.ArcGIS.Geodatabase
Imports ESRI.ArcGIS.Geometry
Imports ESRI.ArcGIS.Carto
```

Step 4: The following is a custom procedure that performs raster classification. Read through the code and comments of the sub procedure and then enter it into the ReclassifySlopeButton class.

```
Private Sub ReclassifyRaster (inputRaster As IRaster2, outFolder As String,
                              outRasterName As String, outFormat As String)
    ' 1. validate user input
    If Not IO.Directory.Exists(outFolder) Then
        MsgBox("Output workspace does not exist.") : Return
    End If
    If IO.File.Exists(outRasterName) Then
        MsgBox("Output raster name already exists.") : Return
    End If
```

```
' 2. save the input raster dataset to the output workspace
'    with an output raster name,  and then get the out raster object
' 2.1 get the output workspace
Dim wksFact As IWorkspaceFactory = New RasterWorkspaceFactory
Dim outWS As IRasterWorkspace2 = wksFact.OpenFromFile(outFolder, 0)

' 2.2 QI to the ISaveAs2 interface of the input raster
Dim rasterSaveAs As ISaveAs2 = inputRaster     ' QI

' 2.3 get ready to save the input raster dataset as an output raster
Dim outRasterDS As IRasterDataset2 = Nothing
Try     ' put the SaveAs operation in a Try block in case it should go wrong
    outRasterDS = rasterSaveAs.SaveAs(outRasterName, outWS, outFormat)
Catch ex As Exception
    MsgBox(ex.Message) : Return
End Try

' 3 get the output raster from the new raster dataset
Dim outRaster As IRaster2 = outRasterDS.CreateFullRaster()

' QI to three interfaces of the output raster for use later
Dim outRasterEdit As IRasterEdit = outRaster
Dim rasterProps As IRasterProps = outRaster

' 4. get raster cursor, and then get pixel blocks from the cursor
' 4.1 define pixel block size.
Dim size As IPnt = New Pnt      ' create a new Pnt object for pixel block size
size.SetCoords(rasterProps.Width, 3)  ' the raster's width x 3 pixels in height

' 4.2 create raster cursors from the output raster
'     a raster cursor is a collection of pixel blocks cut from the raster.
Dim RasterCursor As IRasterCursor = outRaster.CreateCursorEx(size)

' 4.3 declare some variables for use in the repetition block
Dim pixelBlock As IPixelBlock3      ' pixel block
Dim pixels As System.Array          ' safe array of pixels of a block
Dim value As Single                 ' a pixel value

' 4.4 iterate through pixelblocks
Do
    pixelBlock = RasterCursor.PixelBlock    ' gets a pixel block from the cursor

    ' read values of band 0 (the only band) in the pixel block into a safe array
    pixels = CType(pixelBlock.PixelData(0), System.Array)

    ' process each pixel value in the pixel block
    For col As Long = 0 To pixelBlock.Width - 1    ' iterates through columns
        For row As Long = 0 To pixelBlock.Height - 1  ' iterates through rows
            value = Convert.ToSingle(pixels.GetValue(col, row))
            ' reclassify the value if it is not NoData and update the pixel
            If value <> Single.MinValue Then
                pixels(col, row) = Classify(value)
            End If
        Next
    Next
    ' set the updated array back to the band of the output pixel block
    pixelBlock.PixelData(0) = pixels
```

```
            ' after processing a pixel block, write the updated block into the raster.
            '   first, get the (col, row) position of the top-left corner
            '   of the current pixel block in the raster
            Dim tlc As IPnt = RasterCursor.TopLeft
            outRasterEdit.Write(tlc, pixelBlock)  ' writes the block into the out raster
        Loop While RasterCursor.Next

        ' recycle memory
        System.Runtime.InteropServices.Marshal.ReleaseComObject(inputRaster)
        System.Runtime.InteropServices.Marshal.ReleaseComObject(outRaster)
        MsgBox("Raster successfully reclassified!")
    End Sub
```

Step 5: Finally, implement the OnClick event procedure of ReclassifySlopeButton.

```
    Protected Overrides Sub OnClick()
        Dim mxDoc As IMxDocument = My.ArcMap.Document
        Dim lyr As ILayer = mxDoc.SelectedLayer     ' gets the selected layer
        If lyr Is Nothing Then
            MsgBox("Please select a slope raster layer.") : Return
        End If
        Dim rlyr As IRasterLayer = TryCast(lyr, IRasterLayer)   ' QI
        If rlyr Is Nothing Then
            MsgBox("The selected layer is not a raster.") : Return
        End If

        ' get the raster object from the raster layer
        Dim raster As IRaster2 = rlyr.Raster

        ' call your ReclassifyRaster () sub procedure.
        ReclassifyRaster (raster, "C:\Temp", "slopeclasses", "GRID")
    End Sub
```

To test the slope reclassification capability, you may load the slope raster named “slope” in the Data folder on the CD-ROM. Make sure the output folder (C:\Temp) exists on your computer and then run the program. If nothing goes wrong, the resulting raster should look similar to Figure 10 when slope classes are properly rendered in ArcMap.

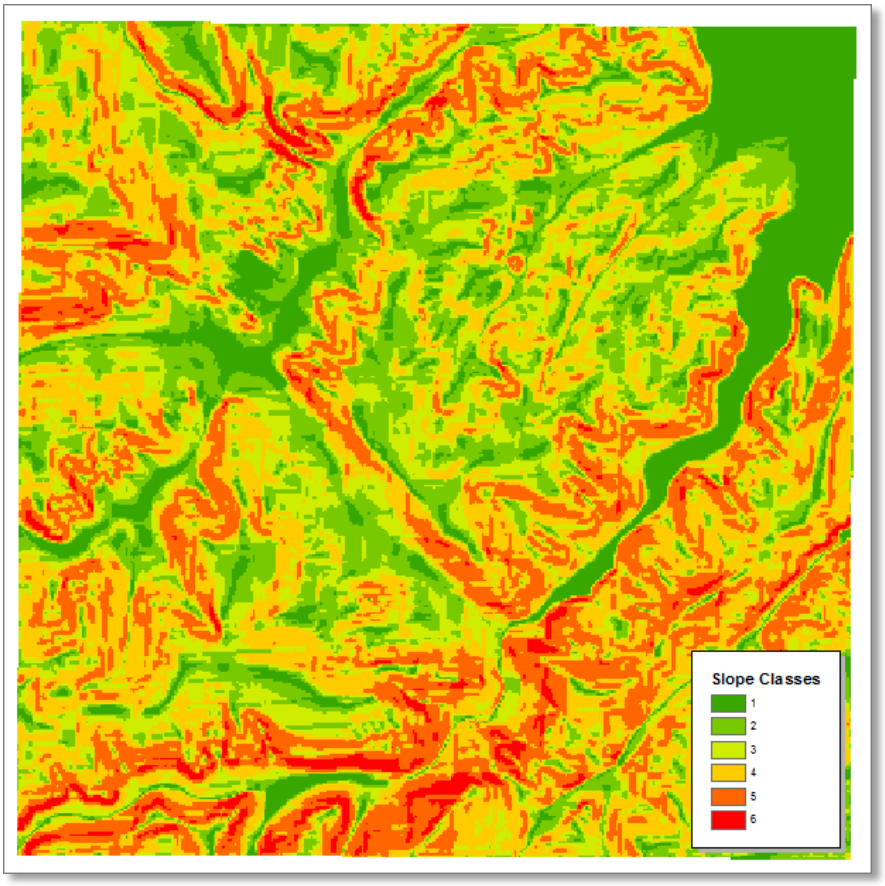

Figure 10 Resulted slope class raster

4. Focal Function – Slope Raster Generation

Example 4: Create an add-in button that generates a slope raster from a DEM.

4.1 Slope algorithms

To generate a slope raster from a DEM you need an algorithm. In this example, you will use the *third-order finite difference algorithm* to compute slope gradient. Given a 3 x 3 matrix of cells each containing an elevation value, the third-order finite difference algorithm calculates the slope of the central cell. The Figure 11 illustrates how the algorithm works.

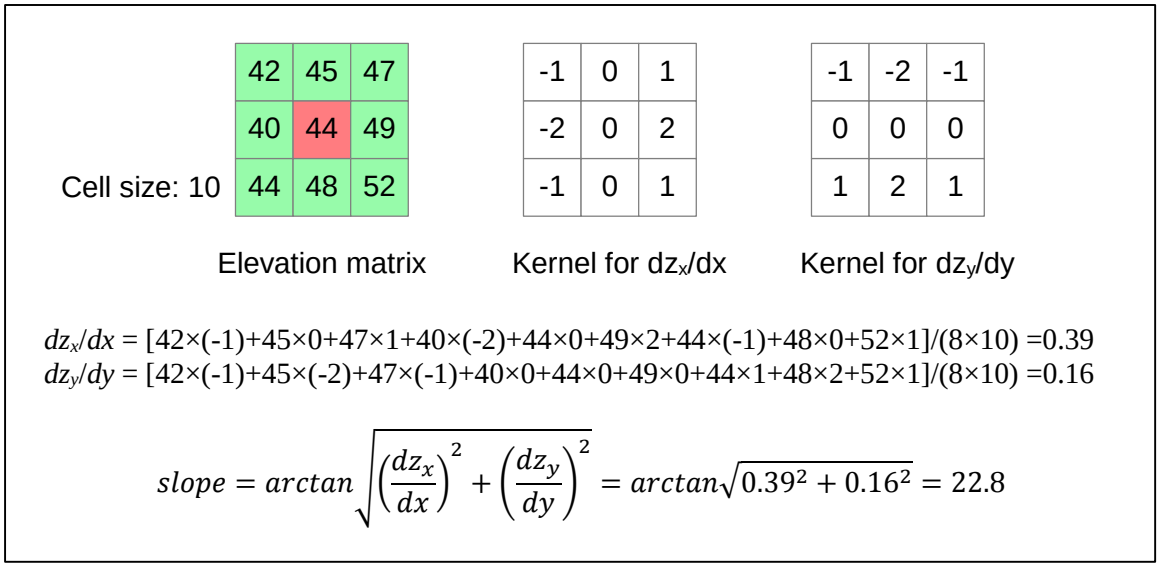

Figure 11 The third-order finite difference algorithm for slope gradient

The algorithm uses two kernels to calculate two slope parameters dz_x/dx and dz_y/dy. A kernel is a matrix containing constant weight values. A slope parameter is the weighted average slope between each pair of adjacent cells in a specified direction (x or y direction). It is calculated by multiplying each elevation value by a corresponding weight in a kernel, and then dividing the total distance between cells in the specific direction (cell size x 8). A slope gradient (in degrees) is finally calculated by the formula shown in Figure 11. Given a 3 x 3 processing window represented by a 2-D array of elevation values, a pixel width and pixel height, the third-order finite difference algorithm can be implemented as the following:

```
Private Function ThirdOrderFiniteDiffSlope(procwin As Single(,),
                    pixelWidth As Double, pixelHeight As Double) As Double
    ' create a 2-D array for the 3 x 3 kernel for dz/dx
    Dim kernelx(,) As Integer = {{-1, -2, -1}, {0, 0, 0}, {1, 2, 1}}
    ' create a 2-d array for the 3 x 3 kernel for dz/dy
    Dim kernely(,) As Integer = {{-1, 0, 1}, {-2, 0, 2}, {-1, 0, 1}}
    Const RAD As Double = 180 / Math.PI  ' radian to degree factor
    ' Note: for computation efficiency, these two array variables and
    ' the RAD constant can be declared at class-level

    ' if the central cell contains NoData, return NoData without computation
    If procwin(1, 1) = NoData(0) Then
        Return NoData(0)
    End If

    Dim dzdx As Double          ' dz/dx
    Dim dzdy As Double          ' dz/dy
    Dim cwx As Integer = 0      ' cumulative weights in x direction
    Dim cwy As Integer = 0      ' cumulative weights in y direction
    For i As Integer = 0 To 2       ' iterates through all columns
        For j As Integer = 0 To 2   ' iterates through all rows
            If procwin(i, j) <> NoData(0) Then
                dzdx += procwin(i, j) * kernelx(i, j)
                dzdy += procwin(i, j) * kernely(i, j)
                cwx += Math.Abs(kernelx(i, j))
```

```
                cwy += Math.Abs(kernely(i, j))
            End If
        Next
    Next

    dzdx /= (pixelWidth * cwx)
    dzdy /= (pixelHeight * cwy)

    Dim slope As Double = Math.Atan(Math.Sqrt(dzdx ^ 2 + dzdy ^ 2)) * RAD
    Return slope
End Function
```

The ThirdOrderFiniteDiffSlope() function has three parameters. The first parameter *procwin* is a Single type two-dimensional array. It is the processing window that contains 3 column x 3 row elevation values. The second and third parameters are used to receive pixel width and height. After computation, the function returns a Double type slope value. Inside the nested repetition block, array variable *NoData* is a class-level variable carrying a no-data value for each band of the raster. NoData(0) is the no-data value of the only band of a DEM. You will declare this class-level array variable and assign a value for it later.

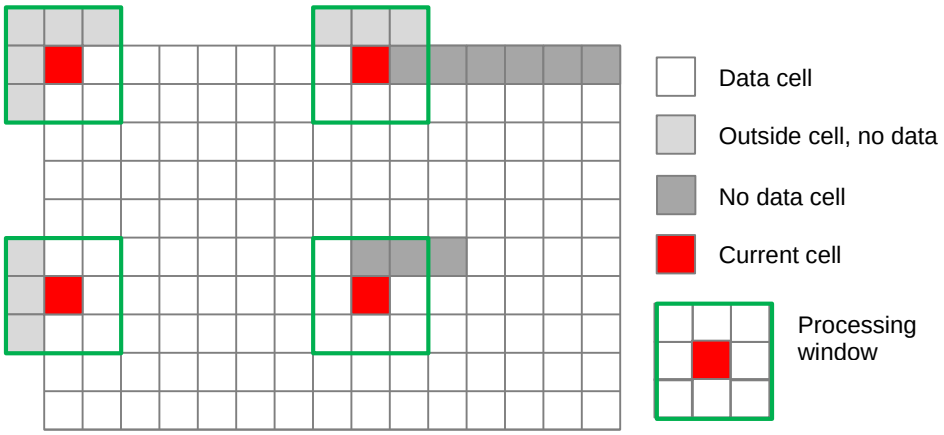

Figure 12 Exceptions in processing window size and the current pixel position

Using the third-order finite difference algorithm to calculate a central cell within a regular 3 x 3 window is straightforward. But practice is often more complex than theory. While computing slope gradients from a large DEM by moving a processing window across the DEM column by column and row by row, you can encounter various exceptions (Figure 12). For example, while processing a cell on a border or corner, some cells of the processing window can extend out of the raster, therefore they contain NoData values. Even in the middle of a raster, the processing window may encounter NoData cells. If a processing window contains no-data cells, the third-order finite difference slope algorithm may produce a wrong result or even crash. Handling no-data cells will be discussed in the following two sub-sections.

4.2 No data value

No data value is a special value used to indicate invalid or excluded data in a raster. Please recall that when you create a new raster dataset in Example 2, you set 255 as no data value to the NoDataValue property on the IRasterProps interface (Figure 13). The NoDataValue property is a two-way property of Variant type. Variant indicates that it is not a fixed data type.

When you set a value to it, the value should be the same type as the raster's pixel data. For example, the pixel data type of a DEM is usually Single, therefore the no-data value you set to the NoDataValue property of a DEM must also be a Single value. The minimum value of the Single data type (−3.4028235E+38, represented by VB constant Single.MinValue) is commonly used in this case. Images (such as aerial photos) often use 1-byte integer for pixel values, and the maximum value of 1-byte integer (255) is often used to indicate no-data for images. In ArcGIS, no data cells of a raster are displayed as transparent.

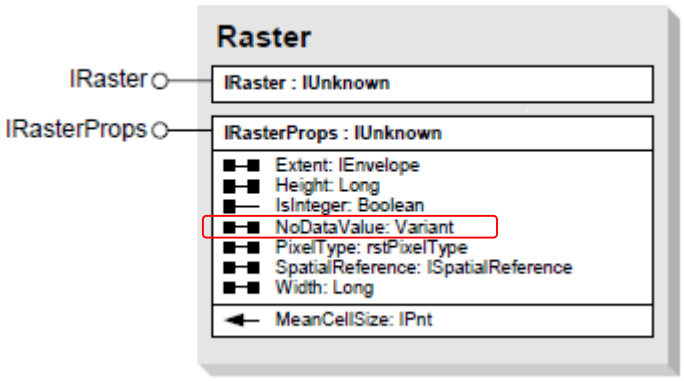

Figure 13 The IRasterProps interface

When you get a value from the NoDataValue property, you get an array object. The data type of the array is same as the pixel data type of the raster, and the size (length) of the array is the number of bands. Each element in the array holds a no-data value for a band. Suppose you have a raster object represented by variable *raster* pointing to IRaster interface, you can find its pixel data type from the PixelType property on the IRasterProps interface. Suppose the pixel data type of the raster is Single, you can use the following code to get the no-data value of the raster.

```
    Dim rstProps As IRasterProps = raster          ' QI to IRasterProps interface
    Dim NoData() As Single = rstProps.NoDataValue  ' gets the NoDataValue array object
```

The no-data value of the top band is held by NoData(0).

4.3 Getting processing windows from a pixel array

You know that a pixel array can be retrieved from a raster dataset after hopping a number of steps (raster dataset –> raster –> raster cursor –> pixel block –> pixel array). A pixel array is a two-dimensional array of pixel values. In Example 2, you used two nested For … Next loops to iterate through all columns and rows and performed slope reclassification for each pixel. To compute slope of a pixel or cell, a window of cells surrounding the current cell must be create and passed to the ThirdOrderFinitDiffSlope() function because the slope algorithm is a focal function. Given an array of pixels derived from a pixel block that covers the entire raster and the column and row numbers of a currently processing cell in the raster, the following function creates a 3 x 3 cell processing window.

```
Private Function GetProcWindow(data As Single(,),
                               col As Integer,
                               row As Integer) As System.Array
    ' declare a 3 x 3 array variable, same data type as the raster
```

```
    Dim win(2, 2) As Single

    ' if the current cell contains no data, no slope will be calculated,
    ' let the central cell of the processing windows to be NoData and return result
    If data(col, row) = NoData(0) Then
        win(1, 1) = NoData(0)
        Return win
    End If

    ' read cell values from the pixel array into the processing window
    For i As Integer = -1 To 1
        For j As Integer = -1 To 1
            If col + i < 0 Or col + i > data.GetUpperBound(0) Then
                ' current cell is on the left or right border
                win(i + 1, j + 1) = NoData(0)
            ElseIf row + j < 0 Or row + j > data.GetUpperBound(1) Then
                ' current cell is on the top or bottom border
                win(i + 1, j + 1) = NoData(0)
            Else ' current cell in middle of the raster
                win(i + 1, j + 1) = data(col + i, row + j)
            End If
        Next
    Next

    ' Handle NoData cells in the processing window on edges:
    ' if a cell contains NoData but the rotationally symmetrical cell
    ' contains a value, calculate a value for the NoData cell in a way so that
    ' the overall slope trend in the processing window is preserved.
    Dim oi, oj As Integer       ' for opposite column and row
    For i As Integer = 0 To 2
        For j As Integer = 0 To 2
            If i = 1 And j = 1 Then Continue For  ' the central cell
            oi = 2 - i          ' opposite column
            oj = 2 - j          ' opposite row
            If win(i, j) = NoData(0) Then
                If win(oi, oj) <> NoData(0) Then
                    win(i, j) = win(1, 1) * 2 - win(oi, oj)
                End If
            End If
        Next
    Next

    Return win      ' returns the processing window
End Function
```

This GetProcWindow() function has three parameters. The first one (data As Single(,)) is declared as a two dimensional array of single data type values. The remaining two arguments specify the column and row position of the currently processing cell. Inside the function, a 3 x 3 Single type array is declared as the output processing window. If the current cell contains no data, it assigns a no-data value to the central cell of the processing window and immediately returns the result. If the current cell contains a valid value, values surrounding the current cell are read from the input pixel array (parameter *data*) into the processing window. Once populated, the processing window can still contain no-data values if the current cell is an edge cell. In this case, a new value is calculated for a no-data value cell based on the central cell and the rotationally symmetrical cell in the processing window. After this handling, the third-order finite difference algorithm function becomes reliable.

4.4 Overall procedure

With these two key functions created, you are ready to implement the slope computation add-in button. The following is an overall procedure for generating a slope raster from a DEM raster (Figure 14).

1) Save the DEM as a new raster dataset with the output (slope) dataset name, i.e. to make a copy of the DEM with the output dataset name. Although you could create a blank raster from scratch, making a copy of the input DEM and use it as an output slope raster is much easier.
2) Get a raster from the slope dataset. It still contains elevation values.
3) Create a raster cursor from the DEM raster (by using the full size of the raster).
4) Get a pixel block from the raster cursor (this pixel block covers the entire raster).
5) Get a pixel array from the pixel block. Pixel values are elevations at this point.
6) Iterate through all pixels in the pixel array to compute slopes and store results in a new slope array.
7) Update the slope array into the pixel block. Pixel values in the block become slopes after this step.
8) Save the slope pixel block back into the slope raster to update the slope dataset.

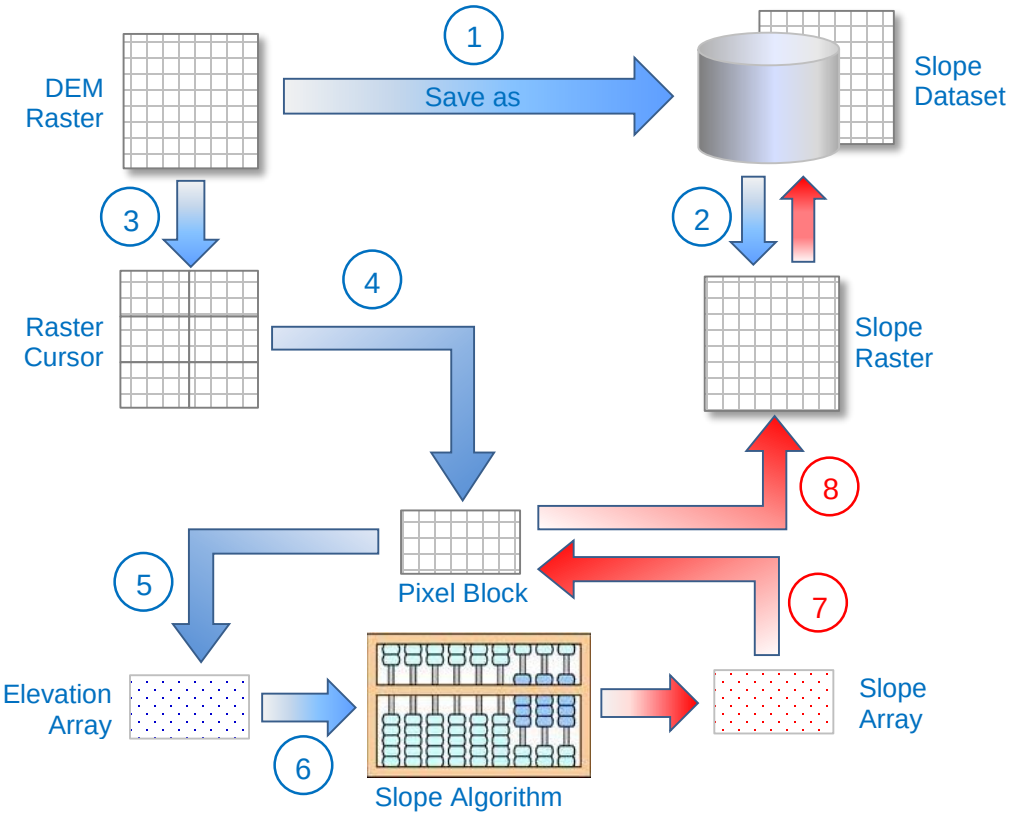

Figure 14 Procedure of slope raster generation

These eight steps can be implemented by a sub procedure as the following:

```
Private Sub CreateSlopeRaster(demRaster As IRaster2,
```

```
                                outFolder As String, slopeRasterName As String)
' 1. Call a custom function to save the input DEM as a new raster dataset
Dim slopeRasterDS As IRasterDataset2 =
    SaveRasterAs(demRaster, outFolder, slopeRasterName)

' 2. create a raster object from the slope raster dataset
Dim slopeRaster As IRaster2 = slopeRasterDS.CreateFullRaster

' 3. create a raster cursor from the slope raster containing elevations at this point
' 3.1 find the raster size (number of columns and rows)
Dim size As IPnt = New Pnt                    ' a Pnt object for pixel block size
Dim rstProps As IRasterProps = slopeRaster  ' QI to IRasterProps interface
Dim cols As Integer = rstProps.Width        ' number of columns
Dim rows As Integer = rstProps.Height       ' number of rows
size.SetCoords(cols, rows)                  ' sets up the size object
NoData = rstProps.NoDataValue               ' gets the NoDataValue array object

' 3.2 create raster cursors from the DEM raster with the intended pixel block size
Dim rasterCursor As IRasterCursor = demRaster.CreateCursorEx(size)

' 4. get a pixel block from the raster cursor with the specified block size
Dim PixelBlock As IPixelBlock3 = rasterCursor.PixelBlock

' 5. read values of a pixel block into a safe array, values are elevations
Dim demPixels As System.Array = CType(PixelBlock.PixelData(0), System.Array)

' declare a 2-d array of the same size for output slopes
Dim slopePixels(cols - 1, rows - 1) As Single

' 6. Compute slope values for pixels
' 6.1 get pixel size of the raster. This is needed by the slope algorithm
Dim rstAnalysisProps As IRasterAnalysisProps = slopeRaster ' QI
Dim pixelWidth As Double = rstAnalysisProps.PixelWidth
Dim pixelHeight As Double = rstAnalysisProps.PixelHeight

' 6.2 declare a 2-D array variable as a processing window
Dim win(,) As Single

' 6.3 calculate slope pixel by pixel
For i As Long = 0 To cols - 1       ' iterate through columns
    For j As Long = 0 To rows - 1   ' iterate through rows
        ' get a processing window around the current pixel
        win = GetProcWindow(demPixels, i, j)
        ' calculate slope for the current pixel in the window
        slopePixels(i, j) = ThirdOrderFiniteDiffSlope(win, pixelWidth, pixelHeight)
    Next
Next

' 7 set slope pixel array to the top band of the pixel block
PixelBlock.PixelData(0) = slopePixels

' 8. save output pixel block to target raster dataset
' 8.1 QI to the IRasterEdit interface of the target raster for writing data
Dim slopeRasterEdit As IRasterEdit = slopeRaster

' 8.2 write the pixel block into the target raster
Dim tlc As IPnt = New Pnt()    'demRasterCursor.TopLeft
tlc.SetCoords(0, 0)
```

```
    ' 8.3 save the pixel block
    slopeRasterEdit.Write(tlc, PixelBlock)

    ' clean up
    System.Runtime.InteropServices.Marshal.ReleaseComObject(slopeRaster)
    System.Runtime.InteropServices.Marshal.ReleaseComObject(demRaster)
    MsgBox("Slope raster has been successfully created!")
End Sub
```

Step 1 in the above procedure is carried out by a custom function SaveRasterAs() with the following code.

```
Private Function SaveRasterAs(raster As IRaster2,
                              folder As String,
                              newName As String) As IRasterDataset2
    ' create a workspace factory from the rasterworkspace factory co-class
    Dim wksFact As IWorkspaceFactory = New RasterWorkspaceFactory()
    ' open the user specified workspace
    Dim ws As IRasterWorkspace = wksFact.OpenFromFile(folder, 0)

    Dim format As String = raster.RasterDataset.Format  ' gets the source raster format
    Dim rasterSaveAs As ISaveAs = raster  ' QI to ISaveAs interface of the source raster
    Dim newRasterDS As IRasterDataset2 = rasterSaveAs.SaveAs(newName, ws, format)
    Return newRasterDS      ' returns the new raster dataset
End Function
```

4.5 Final assembly

Now, you have almost all code needed to carry out each step and implement the overall functionality. Please create an add-in button named “ComputeSlopeButton” in your Raster Analyst project. Use caption and tip text “Create Slope Raster” and icon:  (in folder Images on CD-ROM). Add the button into the project toolbar after you create it. Then perform the following steps.

1) Add the following import statements in the ComputeSlopeButton class module.

```
Imports ESRI.ArcGIS.ArcMapUI
Imports ESRI.ArcGIS.Carto
Imports ESRI.ArcGIS.Geometry
Imports ESRI.ArcGIS.Geodatabase
Imports ESRI.ArcGIS.DataSourcesRaster
```

2) Declare a class-level private array variable in the ComputeSlopeButton class for no-data value of the slope raster. This array will be assigned value later in sub procedure CreateSlopeRaster().

```
Private NoData() As Single
```

3) Enter the following procedures into the ComputeSlopeButton class:

```
ThirdOrderFinitDiffSlope()
GetProcWindow()
```

```
SaveRasterAs()
CreateSlopeRaster()
```

4) Implement the OnClick event procedure of the ComputeSlopeButton.

```
Dim mxDoc As IMxDocument = My.ArcMap.Document     ' gets the map document
Dim lyr As ILayer = mxDoc.SelectedLayer           ' gets the selected layer
If lyr Is Nothing Then
    MsgBox("Please select a DEM layer") : Return  ' if no selected layer
End If

' QI to IRasterLayer
' Use function TryCast to avoid crashing when a non-raster layer is selected
Dim rlyr As IRasterLayer = TryCast(lyr, IRasterLayer)
If rlyr Is Nothing Then
    MsgBox("Please select a DEM layer") : Return
End If

' get a raster object from the raster layer
Dim demRaster As IRaster2 = rlyr.Raster

' call the CreateSlopeRaster sub procedure
CreateSlopeRaster(demRaster, "C:\Temp", "Slope")
```

Figure 15 demonstrates how procedures are called in the process.

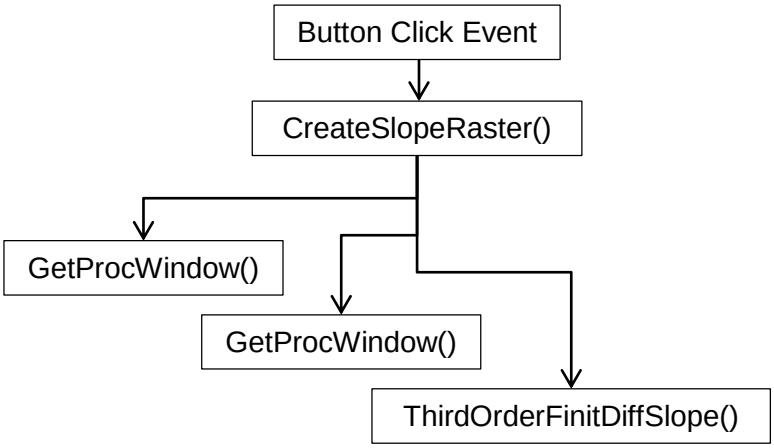

Figure 15 Procedure calls in slope raster generation

To test the slope computation add-in button, please start the program and load the dem in the Data folder on the CD-ROM. If your program runs successfully, a resulted slope raster should be similar to the screenshot shown in Figure 16 after stretching with standard deviation. If you compare this slope raster with a result produced by the slope tool in the ArcToolbox, you will find both are almost identical. With a careful examination of the two slope rasters, you will find this tool outperforms the built-in tool in edge handling. Besides that, this slope tool is much faster than the geoprocessing tool.

The program is not perfect at this point since the input DEM and output slope raster names are hard coded. I hope you can create a friendly user interface for the tool to make it a professional product.

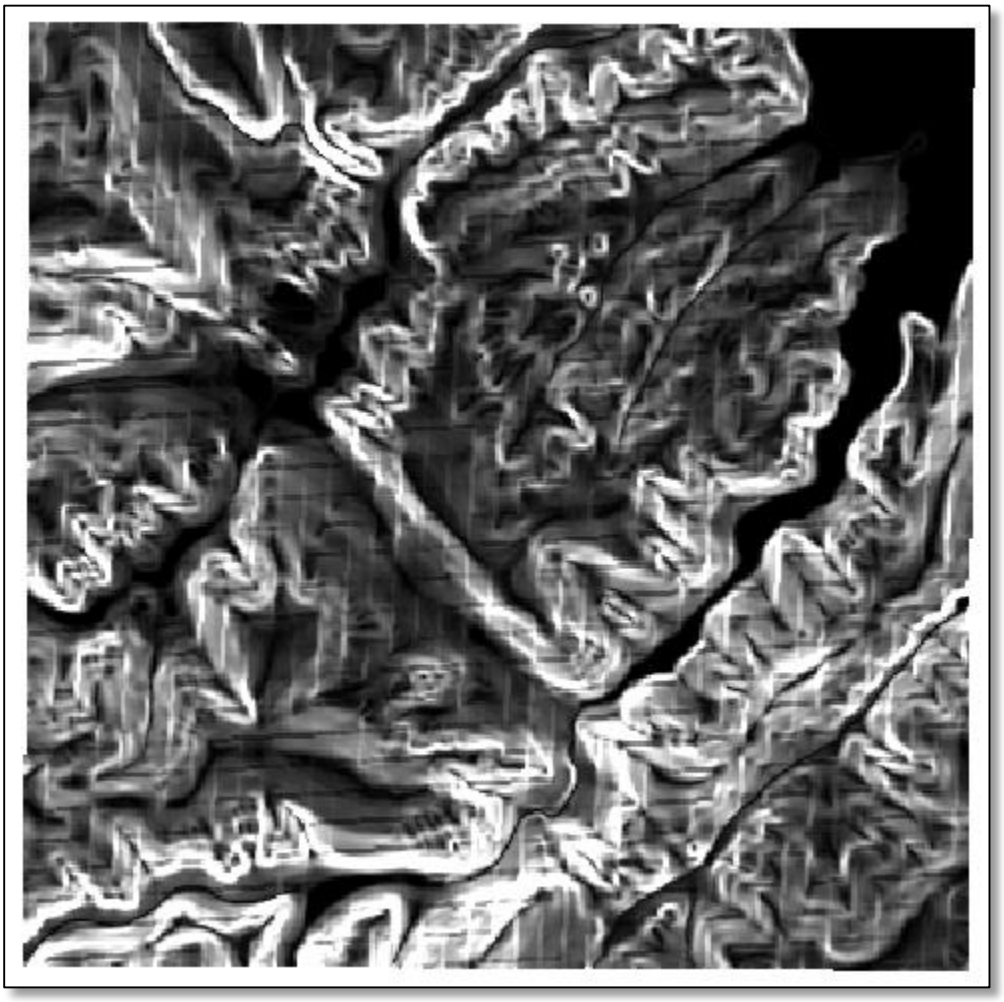

Figure 16 Slope raster produced by the add-in slope tool

Summary and Highlights

A raster dataset object represents a set of raster data stored in a file system or in geodatabase on the hard drive. A raster object is the pixel data loaded into the computer memory.

The Raster co-class has many interfaces managed by different namespaces.

1) The IRaster interface contains methods for creating raster cursors and creating pixel blocks.
2) The IRaster2 interface provides methods for getting pixel values, creating pixel cursors, as well as converting between raster column and row positions to map coordinates.
3) The IRasterProps interface contains properties that describe the raster. These include raster extent, width and height (i.e. columns and rows), pixel type, NoData value, and spatial reference.
4) The IRasterBandCollection interface provides methods for managing bands, such as adding and removing bands.
5) The IRasterEdit property provides a method for writing pixel blocks into the raster.

A pixel block is a block of pixels retrieved from a raster object using a user specified size. Pixel blocks are used to facilitate data retrieval and processing for large raster data sets. A pixel block may be composed of multiple bands. Each band in a pixel block is an array of values.

New pixel blocks can be created by the CreatePixelBlock() method on the IRaster interface. To access existing raster data, a pixel cursor can be created by calling the CreateCursorEx() method on the IRaster2 interface. A pixel cursor is a collection of pixel blocks cut from the raster. Individual pixel blocks can be retrieved from a pixel cursor. To access pixel values in a raster dataset, you usually need to make a series of hops: raster dataset –> raster –> pixel cursor –> pixel block –> value array of a band –> a pixel value.

To write values into a raster, values are first written in a 2-dimensional array which represents a band in a pixel block. Then the array is assigned to the PixelData property of the pixel block object with a specific band, and the pixel block is written into a raster by calling the Write() method on the IRasterEdit interface.

Exercises

1. Answer the following questions.
   1) What is a raster dataset?
   2) How to obtain a raster dataset object from a given file path.
   3) What is a raster?
   4) What are differences between a raster dataset and a raster?
   5) How can you create a raster from a raster layer?
   6) Why is a raster compared to a feature class in the chapter?
   7) How many different interfaces of the raster class can you name? what are they?
   8) What method on which interface of a raster can you use to get pixel values of given map coordinates?
   9) What methods on which interface of the raster class can be used to convert between map coordinates and raster pixel locations in columns and rows?
   10) What method on which interface of the Workspace class can be used to create new raster datasets?
   11) The CreateRasterDataset method requires 11 arguments in order to create a new raster dataset. Can you briefly explain each of these arguments?
   12) What raster formats does the CreateRasterDataset method support?
   13) If you want to create a raster dataset for about 20 different land cover types, what pixel type would you choose?
   14) If you want to create a raster dataset for slope gradient, what pixel type would you select?
   15) If you want to create a raster dataset for a scanned color aerial photo, what pixel type would you use?
   16) How method on which interface of raster dataset do you use create a raster object?
   17) What is a pixel block?
   18) How can a pixel block created from a raster?
   19) A pixel block contains only one band. True or false?
   20) A pixel block is always a square with same number of columns and rows. True or false?
   21) What is a two-dimensional array?
   22) How many arrays does a pixel block contain?
   23) What is a safe array?
   24) Why do we use a nested repetition blocks to access values from a pixel array?
   25) What property on the pixel block object do you set a pixel array to?
   26) What method on which interface of a raster object do you use to save a pixel block into the dataset?

2. Since you can find pixel values of one location, you should be able to get pixel values of multiple locations. Suppose you have a point shapefile in which each point feature represents a sampling location, can you figure out a procedure to extract the elevation from a DEM for each sampling location and save results in the attribute table of the shapefile?

3. Improve the CreateRasterButton in Example 2 by creating a Windows dialog (form) for the user to enter or select argument values for the new raster dataset. The form should allow the user to select a workspace (a file / personal geodatabase or a folder), enter a raster name, select raster format including ESRI grid, TIFF, BMP, Imagine, and Idrisi, as well as to specify other raster specifications such as spatial reference. Use this form as a dialog to collect argument values from the user when the add-in button is clicked. You need to create

a property for the Widows dialog (form) class to hold each piece of data collected in the form class in order to return values to the client.

4. In sub procedure CreateSlopeRaster() in Example 4, the first step creates a raster dataset for output by copying the DEM raster to an output workspace. Please modify the code for this step to create a blank raster dataset from scratch and use it for slope computation output.